

Python marshal 模块稳定性与正确性测试报告

仓库链接: <https://github.com/cccbbaaa666/marshal-stability-suite>

1. 目标

本作业的目标是评估 Python marshal 模块的稳定性与正确性, 重点回答:

- 相同输入在相同 Python 版本内是否总能产生 hash-identical 的字节流。
- 不同平台、不同 Python 版本、浮点特殊值、递归结构与大规模集合是否会影响结果。
- dump / dumps 与 load / loads 在正常与异常输入下是否表现正确。

marshal 官方文档明确说明格式故意不保证跨 Python 版本稳定, 因此本测试套件把“跨版本字节完全一致”视为调查项, 而不是必须通过的断言。

2. 测试套件设计

本仓库采用 unittest, 仅依赖标准库, 便于本地运行和 GitHub Actions 矩阵执行。测试文件如下:

- tests/test_roundtrip.py: 基础正确性、文件 API、代码对象、递归与共享引用
- tests/test_determinism.py: 同进程与跨进程稳定性、不同 PYTHONHASHSEED、NaN 载荷
- tests/test_boundaries.py: 边界值分析, 覆盖整数、长度阈值、深度阈值、大集合
- tests/test_errors.py: 非法输入、截断字节流、无效 tag、尾随垃圾字节
- tests/test_fuzzing.py: 固定种子的随机模糊测试
- scripts/collect_marshal_digests.py: 采集当前环境下代表性样例的 SHA-256 摘要

此外, .github/workflows/ci.yml 提供了 Windows、Linux、macOS 与 Python 3.11-3.13 的矩阵执行环境, 用于补充跨平台/跨版本证据。

3. 采用的测试技术

3.1 等价类划分

我将输入划分为以下主要等价类:

- 标量: None、布尔、整数、浮点、复数
- 文本与二进制: str、bytes、bytearray
- 容器: tuple、list、dict、set、frozenset
- 特殊单例: Ellipsis、StopIteration
- 内部对象: 代码对象 code
- 非法输入: object()、lambda、类对象、截断字节流、非法 tag

这样做的原因是 marshal.c 的核心实现是按对象类型分派的, 等价类划分可以较高效地覆盖主要分支。

3.2 边界值分析

边界值分析用于覆盖最容易出现编码变化或实现缺陷的阈值：

- 整数边界：0、-1、`2**31-1`、`-2**31`、`2**63-1`、`-2**63`、`2**1000`
- 长度边界：0、1、255、256、1024
- 容器规模：空集合与数千到上万元素的大集合
- 深度边界：深度 200 的合法嵌套与深度 2500 的过深嵌套
- 版本边界：版本 2/3 对递归结构支持的差异

边界值分析非常适合本题，因为 `marshal` 的实现中存在长度编码、递归深度限制和版本分支。

3.3 模糊测试

我使用固定随机种子的轻量级模糊测试生成 300 个随机合法对象，验证：

- 重复 `marshal.dumps()` 是否稳定
- `loads(dumps(x))` 是否与原值一致

之所以采用固定种子，是为了在保留随机探索能力的同时让结果可复现。没有使用外部模糊测试库，是为了降低依赖并保证 CI 可直接运行。

3.4 白盒测试

虽然作业要求以黑盒为主，但我也依据 `marshal.c` 的已知实现特点补充了白盒导向测试：

- 类型分派分支：覆盖主要支持类型和不支持类型
- 版本分支：版本 2 的集合支持、版本 3 的递归/共享引用支持
- 错误分支：非法 tag、截断输入、对象过深
- 引用处理：别名共享和自引用结构

我没有做严格的 all-definitions/all-uses 覆盖测量，因为本地环境并未直接集成 CPython 源码级覆盖工具；相反，我用“源代码分支特征 -> 测试用例”的映射来证明白盒覆盖思路。

4. 为什么没有采用某些技术

- 没有做基于源码插桩的语句/分支覆盖率统计，因为作业目录不包含可重编译的 CPython，且这会显著增加环境复杂度。
- 没有把“跨 Python 版本字节完全一致”做成必须通过的自动断言，因为这与 `marshal` 官方文档的设计目标相冲突，容易产生伪失败。
- 没有做安全性渗透测试，因为题目关注的是稳定性与正确性，而不是反序列化攻击面。

5. 可追踪性矩阵

需求/风险	测试技术	代表测试
支持类型可正确往返	等价类划分	<code>test_common_values_round_trip_across_all_versions</code>

需求/风险	测试技术	代表测试
集合类型在支持版本中工作正常	等价类划分 + 版本分析	<code>test_set_types_round_trip_from_version_two</code>
代码对象可序列化	等价类划分	<code>test_code_object_round_trip</code>
同一输入重复序列化稳定	稳定性测试	<code>test_repeated_dumps_are_hash_identical</code>
跨进程/哈希种子稳定	稳定性测试	<code>test_cross_process_hash_seed_changes_do_not_change_output</code>
NaN 特殊值稳定	边界值 + 特殊值分析	<code>test_custom_nan_payload_is_stable_for_same_input</code>
递归与共享引用正确处理	白盒 + 边界值	<code>test_recursive_and_shared_references_round_trip</code>
旧版本拒绝递归结构	版本边界分析	<code>test_recursive_values_are_rejected_before_version_three</code>
长度阈值处行为正确	边界值分析	<code>test_string_and_bytes_length_boundaries</code> , <code>test_small_tuple_boundary</code>
超大集合和深层嵌套行为正确	边界值分析	<code>test_empty_and_large_collections</code> , <code>test_excessive_nesting_is_rejected</code>
非法输入被拒绝	错误推测	<code>test_dumps_rejects_unsupported_types</code> , <code>test_load_rejects_invalid_tag</code>
截断与尾随垃圾字节处理合理	错误推测	<code>test_loads_rejects_truncated_streams</code> , <code>test_loads_ignores_trailing_bytes</code>
未预见输入组合	模糊测试	<code>test_seeded_random_fuzzing_for_round_trip_and_determinism</code>

6. 测试结果与发现

本地环境：

- Windows 10
- Python 3.11.7
- `marshal.version = 4`

本地运行结果：25 项测试全部通过。

公开仓库 CI 环境：

- Windows / Ubuntu / macOS
- Python 3.11 / 3.12 / 3.13
- GitHub Actions 已生成 9 份 `marshal-digests-*` artifact

6.1 主要发现

1. 同版本内表现出很强的确定性。

对已测样例而言，相同输入在同一进程内多次 `dumps()`、在不同进程中重复执行、以及在不同 `PYTHONHASHSEED` 下执行时，均得到 hash-identical 的字节流。

2. 递归与共享引用在当前版本中可正确处理。

自引用 list、自引用 dict 和共享别名在当前版本下都能往返恢复；而版本 0-2 会拒绝递归 list/dict，这与版本边界预期一致。

3. 过深嵌套会被拒绝。

深度 2500 的嵌套列表在当前环境下触发 `ValueError: object too deeply nested to marshal`，说明实现包含深度保护。

4. 非法与损坏输入处理基本合理。

不支持类型会触发 `ValueError`；截断字节流会触发 `EOFError/ValueError/TypeError`；`loads()` 会忽略尾随垃圾字节。

5. bytearray 的行为值得单独记录。

在 Python 3.11.7 上，`marshal.dumps(bytearray(...))` 可以成功，但 `marshal.loads()` 返回的是 `bytes`，不是 `bytearray`。这不影响字节内容，但影响返回类型，应在报告中明确说明。

6. 跨平台/跨版本自动化验证已覆盖 Python 3.11-3.13。

公开仓库中的 GitHub Actions 已在 Windows、Ubuntu 和 macOS 上运行 Python 3.11、3.12、3.13 三组版本组合，并成功产出 9 份 digest artifact。此前对 3.9/3.10 的探索性矩阵运行出现失败，因此最终提交版本把自动化验证范围明确收敛为 3.11+。

6.2 摘要证据

`scripts/collect_marshal_digests.py` 在本机采集到的代表性摘要显示：

- `none`：1 字节
- `nan`：9 字节
- `negative_zero`：9 字节
- `recursive_list`：10 字节
- `code`：137 字节

这些结果说明当前解释器对代表性对象的编码是稳定可复现的。

与此同时，公开仓库 CI 已经为以下 9 个环境生成 digest 工件：

- Ubuntu 3.11 / 3.12 / 3.13
- Windows 3.11 / 3.12 / 3.13
- macOS 3.11 / 3.12 / 3.13

这说明测试套件已经真正被部署到多操作系统与多 Python 版本环境中执行，而不仅仅停留在设计层面。

7. 局限性

- 我本地直接验证的是 Windows + Python 3.11.7；其余 3.11-3.13 平台/版本组合通过公开 CI 运行覆盖。
- 测试集虽覆盖了大量代表性输入，但不能证明“所有可能 Python 对象”都稳定。
- 模糊测试是轻量级的，主要目标是发现意外组合，而不是穷尽搜索。
- 没有对 CPython `marshal.c` 做源码级覆盖率度量，因此白盒覆盖是“结构化映射”，不是精确百分比。
- 报告未把跨版本差异设为失败条件，因此跨版本结论仍需要结合 CI 产出的 digest 工件逐项比较。

8. 结论

在当前本地环境中，`marshal` 对已测试的支持类型表现出较好的稳定性与正确性：同一输入通常会产生 hash-identical 输出，递归和共享引用在新版本中工作正常，非法输入也会被显式拒绝。

但 `marshal` 仍然不是通用持久化格式，跨 Python 版本稳定性不应被假定；`bytearray` 被加载回 `bytes` 的现象也表明，某些“支持类型”需要更精细地理解其语义。